

## Case statement Vs IF Statement

```
always @ (*)
begin
  if(sel == 2'b00)
  begin
    OUT = A ;
  end
  else if(sel == 2'b01)
  begin
    OUT = B ;
  end
  else if(sel == 2'b10)
  begin
    OUT = C ;
  end
  else
  begin
    OUT = D ;
  end
end
```

Is the two codes  
having same  
synthesis



NO



```
always @ (*)
begin
  case(sel)
    2'b00 : begin
      OUT = A ;
    end
    2'b01 : begin
      OUT = B ;
    end
    2'b10 : begin
      OUT = C ;
    end
    2'b11 : begin
      OUT = D ;
    end
  endcase
end
```

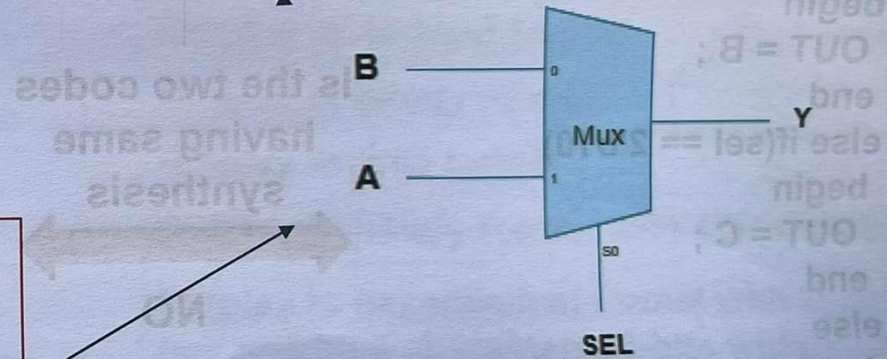
# Conditional Operator

**cond\_expr ? true\_expr : false\_expr**

**assign Y = SEL ? A : B**

**Equivalent**

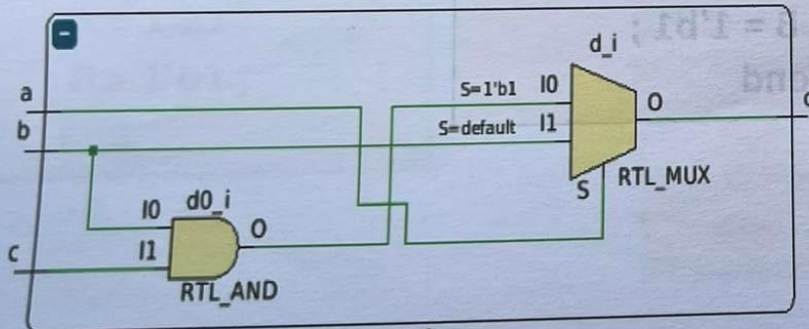
```
always(*)  
  if (SEL)  
    Y = A ;  
  else  
    Y = B ;
```



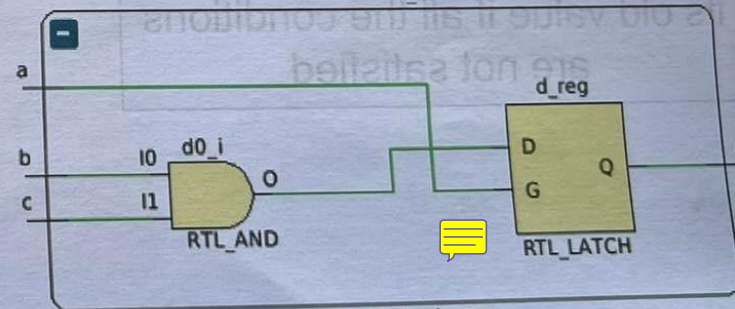
# Unintentional Latch

- In a **combinational** always statement ( always@(\*) ), “if-then-else” and “case” statements must be completely specified for all signals. If not, synthesis tool will generate asynchronous “latches” to retain its old value.

```
always @(*) begin
  if(a) begin
    d = b & c;
  end else begin
    d = b;
  end
end
```



```
always @(*) begin
  if(a) begin
    d = b & c;
  end
end
```



# Unintentional Latch – IF Statement

## Case 1

### “Missing else statement”

If a signal is assigned under different input conditions using if statement and **else statement** is missed, then this signal will infer a latch to retain its old value if all the conditions are not satisfied



```
always @ (*)
if(X)
begin
  A = 1'b1 ;
  B = 1'b0 ;
end
else if (Y)
begin
  A = 1'b1 ;
  B = 1'b1 ;
end
```

**else statement is missed, so A & B signals will infer a latch to retain its values if X & Y conditions are false**

## Unintentional Latch – IF Statement

### Case 1

```
always @ (*)
if(X)
begin
A = 1'b1 ;
B = 1'b0 ;
end
else if (Y)
begin
A = 1'b1 ;
B = 1'b1 ;
end
```

HOW  
TO  
Fix



```
always @ (*)
if(X)
begin
A = 1'b1 ;
B = 1'b0 ;
end
else if (Y)
begin
A = 1'b1 ;
B = 1'b1 ;
end
else
begin
A = 1'b0 ;
B = 1'b0 ;
end
```

Make sure that  
else statement is  
not missed and  
contain all  
variables assigned  
in the  
combinational  
logic blocks under  
different condition  
to avoid  
unintentional latch  
inference

## Unintentional Latch – IF Statement

### Case 2

#### “Incomplete Assignment”

If a signal is assigned under certain input condition using if statement and **not assigned under other input conditions** of the if statement, then this signal will infer a latch to retain its old value under the other input conditions



```
always @ (*)
```

```
if(X)
```

```
begin
```

```
  A = 1'b1 ;
```

```
end
```

```
else if (Y)
```

```
begin
```

```
  B = 1'b1 ;
```

```
end
```

```
else
```

```
begin
```

```
  A = 1'b1 ;
```

```
  B = 1'b0 ;
```

```
end
```

**B signal is missed to be assigned under first if condition and A signal is missed to be assigned under second if condition**

# Unintentional Latch – IF Statement

## Case 2

```
always @ (*)
if(X)
begin
A = 1'b1 ;
end
else if (Y)
begin
B = 1'b1 ;
end
else
begin
A = 1'b1 ;
B = 1'b0 ;
end
```

HOW  
TO  
Fix

```
always @ (*)
if(X)
begin
A = 1'b1 ;
B = 1'b0 ;
end
else if (Y)
begin
A = 1'b0 ;
B = 1'b1 ;
end
else
begin
A = 1'b1 ;
B = 1'b0 ;
end
end
```

Make sure that all variables assigned have an initial or default assignment at the beginning of the always block

OR

Make sure that all variables assigned under all the conditions in the combinatorial blocks

```
always @ (*)
A = 1'b0 ;
B = 1'b0 ;
if(X)
begin
A = 1'b1 ;
end
else if (Y)
begin
B = 1'b1 ;
end
else
begin
A = 1'b1 ;
B = 1'b0 ;
end
```

# Unintentional Latch – Case Statement

## Case 3

### “Missing default branch”

If one or more case branch is missed, then all the signal under all the branches of the case statement will infer a latch to retain its old value when the case condition match one of the missed branches



```
always @ (*)
case (sel)
  2'b00: begin
    A = 1'b1 ;
    B = 1'b0 ;
  end
  2'b01: begin
    A = 1'b0 ;
    B = 1'b1 ;
  end
  2'b11: begin
    A = 1'b1 ;
    B = 1'b1 ;
  end
endcase
```

Not all the case branches of the case statement are mentioned as 2'b10 branch is missed



## Unintentional Latch – Case Statement

### Case 3

```
always @ (*)
case (sel)
  2'b00: begin
    A = 1'b1 ;
    B = 1'b0 ;
  end
  2'b01: begin
    A = 1'b0 ;
    B = 1'b1 ;
  end
  2'b11: begin
    A = 1'b1 ;
    B = 1'b1 ;
  end
endcase
```

HOW  
TO  
Fix



```
always @ (*)
case (sel)
  2'b00: begin
    A = 1'b1 ;
    B = 1'b0 ;
  end
  2'b01: begin
    A = 1'b0 ;
    B = 1'b1 ;
  end
  2'b11: begin
    A = 1'b1 ;
    B = 1'b1 ;
  end
  default: begin
    A = 1'b1 ;
    B = 1'b0 ;
  end
endcase
```

Make sure that  
all the case  
branches are  
mentioned inside  
the case  
statement

OR

Use the default  
branches if not  
all the case  
branches are  
mentioned

```
always @ (*)
case (sel)
  2'b00: begin
    A = 1'b1 ;
    B = 1'b0 ;
  end
  2'b01: begin
    A = 1'b0 ;
    B = 1'b1 ;
  end
  2'b10 : begin
    A = 1'b1 ;
    B = 1'b0 ;
  end
  2'b11: begin
    A = 1'b1 ;
    B = 1'b1 ;
  end
endcase
```

## Unintentional Latch – Case Statement

### Case 4

#### “Incomplete Assignment”

If a signal is assigned under certain branch using case statement and **not assigned in the other branches**, then this signal will infer a latch to retain its old value when the case condition match the other branches

```
always @ (*)
case (sel)
  2'b00: begin
    A = 1'b1 ;
    B = 1'b0 ;
  end
  2'b01: begin
    B = 1'b1 ;
  end
  2'b10: begin
    A = 1'b1 ;
  end
default: begin
    A = 1'b1 ;
    B = 1'b0 ;
  end
endcase
```

A signal is missed to be assigned in case (2'b01) and B signal is missed to be assigned in case (2'b10)

# Unintentional Latch – Case Statement

## Case 4

```
always @ (*)
case (sel)
  2'b00: begin
    A = 1'b1 ;
    B = 1'b0 ;
  end
  2'b01: begin
    B = 1'b1 ;
  end
  2'b10: begin
    A = 1'b1 ;
  end
  default: begin
    A = 1'b1 ;
    B = 1'b0 ;
  end
endcase
```

HOW  
TO  
Fix



```
always @ (*)
case (sel)
  2'b00: begin
    A = 1'b1 ;
    B = 1'b0 ;
  end
  2'b01: begin
    A = 1'b0 ;
    B = 1'b1 ;
  end
  2'b10: begin
    A = 1'b1 ;
    B = 1'b0 ;
  end
  default: begin
    A = 1'b0 ;
    B = 1'b0 ;
  end
endcase
```

Make sure that all  
variables assigned  
in the case  
statement have an  
initial or default  
assignment at the  
beginning of the  
always block

OR

Make sure that all  
variables assigned  
under all the  
branches of the  
case statement

```
always @ (*)
begin
  A = 1'b0 ;
  B = 1'b0 ;
  case (sel)
    2'b00: begin
      A = 1'b1 ;
      B = 1'b0 ;
    end
    2'b01: begin
      B = 1'b1 ;
    end
    2'b10: begin
      A = 1'b1 ;
    end
    default: begin
      A = 1'b1 ;
      B = 1'b0 ;
    end
  endcase
end
```

## Unintentional Latch - Summary

- Don't miss the **else** statement in the If - else if statement
- Case statement should have a **default condition** to avoid any missing branch.
- Make sure all variables assigned in a combinatorial logic blocks have an **initial or default assignment**

## Simulation Race

- **Simulation Race** means that same HDL code exhibiting two or more possible behaviors, both of which are correct as per the language interpretation, and the simulator is free to choose any one of these multiple interpretations.
  - Different simulators can give different results
  - Different versions of the same simulator can give different results
- **Simulation Race** is usually encountered in Verilog and System Verilog codes, while VHDL does not have any simulation race.
- **Simulation Race has two different types:-**
  - Read-Write Race
  - Write-Write Race

# Read-Write Race

## 1. Combinational Read-Write Race

```
assign a = b & c;  
  
always @ (b or d)  
if (a)  
    o = b & d;  
else  
    o = b | d;
```

```
always @ (b or c)  
a = b & c;  
  
always @ (b or d)  
if (a)  
    o = b & d;  
else  
    o = b | d;
```

- In the shown above code, **a** is **written** into through the assign statement while being **read** through the always block.
- When b gets updated, it will trigger both the assign statements as well as the always block. The if condition in the always block will see the updated value of a, if the assign statement gets executed first. On the other hand, the if condition will see the old value of a, if the always block gets executed first.
- Depending upon which of the two behaviors is chosen by your simulator, you might get a different behavior. 🗨️

# Read-Write Race

## 1. Combinational Read-Write Race

```
assign a = b & c;  
always @ (b or d)  
if (a)  
    o = b & d;  
else  
    o = b | d;
```

How to fix it



```
assign a = b & c;  
always @ (b or d or a) // or always @ (*)  
if (a)  
    o = b & d;  
else  
    o = b | d;
```

- The solution is very simple. In both the examples, the sensitivity list of the second always block should have **a** in it. This will ensure, as soon as **a** gets updated – the second always block gets triggered and values are re-evaluated with the updated value of a. In the worst case, the second always block might get triggered twice, but the final values would be correct. ☺

# Read-Write Race

## 2. Sequential Read-Write Race

```
always @(posedge clk)
b=c;

always @(posedge clk)
a=b;
```

- At the positive edge of clk, both the always blocks get triggered. So, b is being written into (through the first always block), while it is being read also (through the second always block).
- If the first always block gets triggered before the second one. b gets updated. Then a sees the updated value of b. Thus, in effect, the value of c propagated to a through b.
- If the second always block gets triggered before the first one. In this case, a sees the old value of b, and then, b gets updated.
- So, depending on the sequence chosen by the simulator, the value on a could be either c (i.e. the new value of b), or, the old value of b.

# Read-Write Race

## 2. Sequential Read-Write Race

```
always @(posedge clk)
b = c;
```

```
always @(posedge clk)
a = b;
```

How to fix it



```
always @(posedge clk)
b <= c;
```

```
always @(posedge clk)
a <= b;
```

- The solution is very simple. For a sequential element, we should use a **Non-Blocking Assignment (NBA)**. With an NBA, the Right-Hand-Side is read immediately, but the updating of the Left-Hand-Side happens after all the reads scheduled for the current time have already taken place.



## Write-Write Race

```
always @ (b or c)
if (b !=c)
    err_flag=1;
else
    err_flag=0;

always @ (b or d)
if (b==d)
    err_flag=1;
else
    err_flag=0;
```

- Write-Write race occurs when multiple values are being written into a signal – at the same time.
- When b changes, both the always blocks get triggered. It is possible that in one of the always blocks, **err\_flag** is supposed to take the value of 1, while, in the other, it's supposed to take the value of 0. The value that **err\_flag** finally takes will depend on the sequence in which these always blocks get triggered. The one that triggers later – will determine the final value.
- The race is being created because of the assignments in the two always blocks being made to the same signal.
- A Write-Write race can occur through combinations of **assign/assign statements** or **always/always blocks**.  
-- if two assign statements or two always blocks try to update the same variable.

## Write-Write Race

```
always @ (b or c)
```

```
if (b !=c)
```

```
err_flag=1;
```

```
else
```

```
err_flag=0;
```

```
always @ (b or d)
```

```
if (b==d)
```

```
err_flag=1;
```

```
else
```

```
err_flag=0;
```

How to fix it

```
always @ (b or c or d)
```

```
if ( b !=c || b==d )
```

```
err_flag=1;
```

```
else
```

```
err_flag=0;
```

- The solution here is to combine all the condition that control the value of the **err\_flag** variable in a single always block to avoid **assign the same variable in more than one always block, it will give Errors in the synthesis tools**

## Avoiding Simulation Race

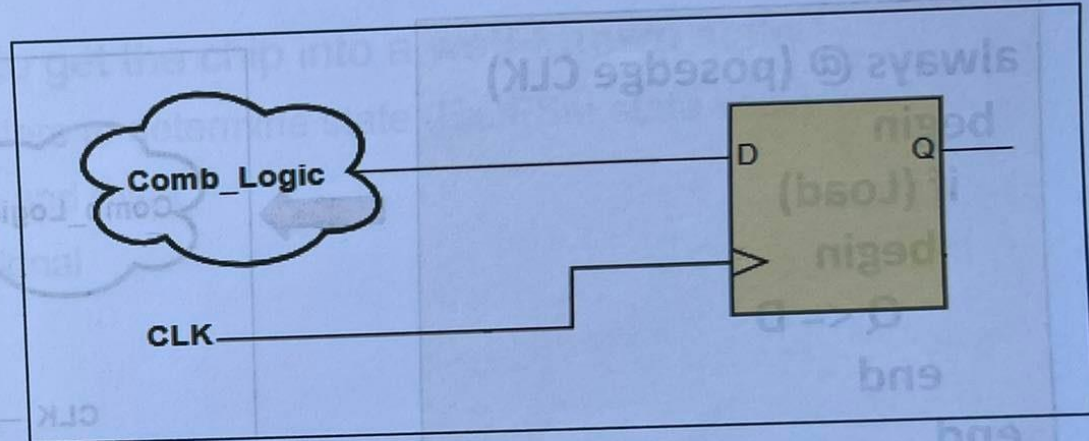
1. When modeling combinational logic with an always block, use blocking assignments.
2. When modeling sequential logic with an always block, use nonblocking assignments.
3. When modeling both sequential and combinational logic within the same always block, use nonblocking assignments.
4. Do not mix blocking and nonblocking assignments in the same always block
5. Do not make assignments to the same variable from more than one always block



# Positive/Negative Edge D Flip Flops

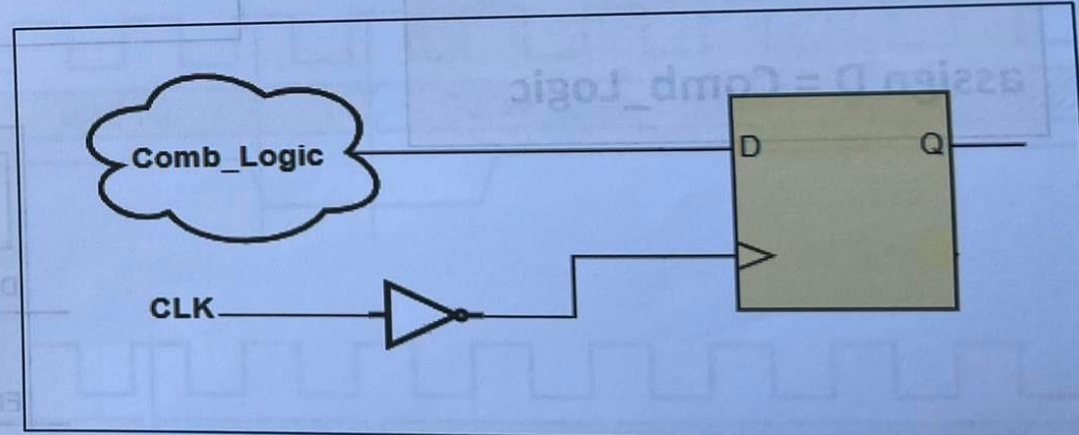
```
always @ (posedge CLK)
begin
  Q <= D
end

assign D = Comb_Logics
```



```
always @ (negedge CLK)
begin
  Q <= D
end

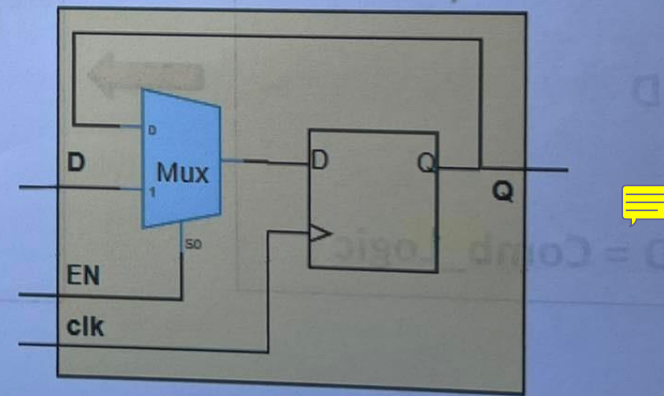
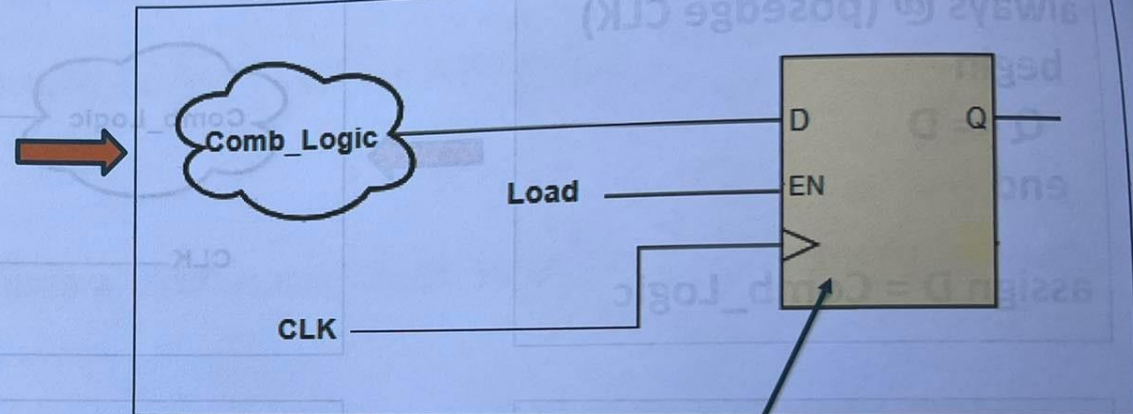
assign D = Comb_Logics
```



# Enabled Positive Edge D Flip Flop

```
always @ (posedge CLK)
begin
    if (Load)
        begin
            Q <= D
        end
    end

    assign D = Comb_Loic
```

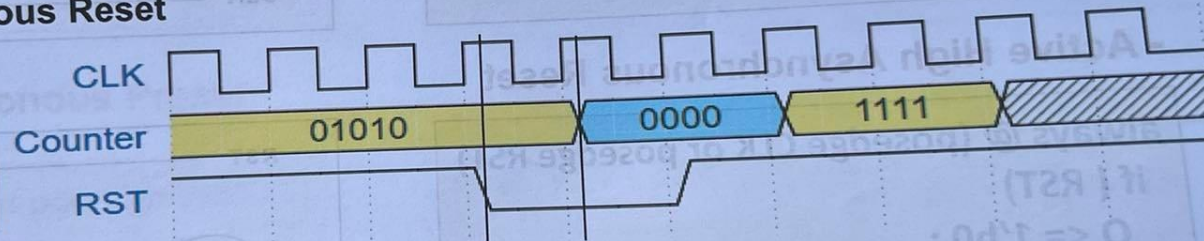


## Flip Flop Reset

- Reset is a global signal intended to get the chip into a **well-known** state.
  - Globally Distributed to registers to determine state (Ex. FSM state vector)
  - Usually asserted on startup and reset
  - Not a designer-controlled signal
- There are two type of resets:-

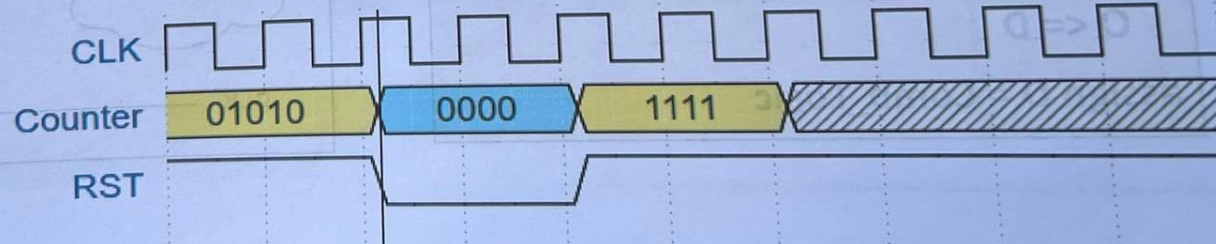
### 1. Synchronous Reset

Active Low  
Synchronous Reset  
Waveform



### 1. Asynchronous Reset

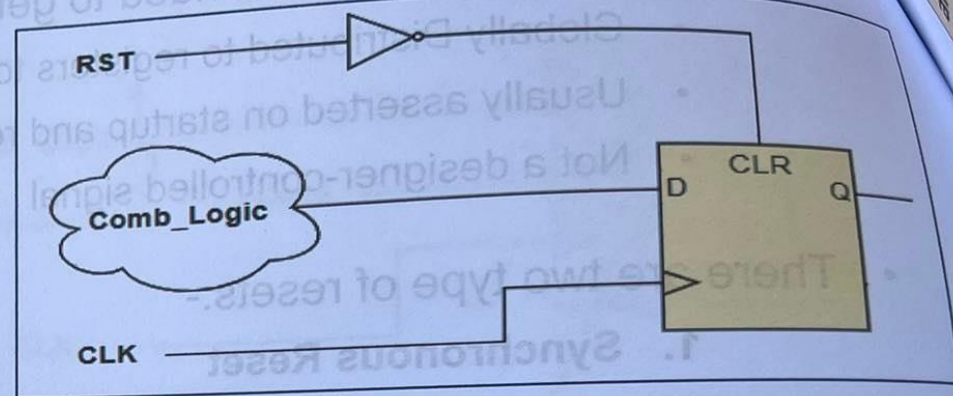
Active Low  
Asynchronous Reset  
Waveform



# D Flip Flop With Asynchronous Reset

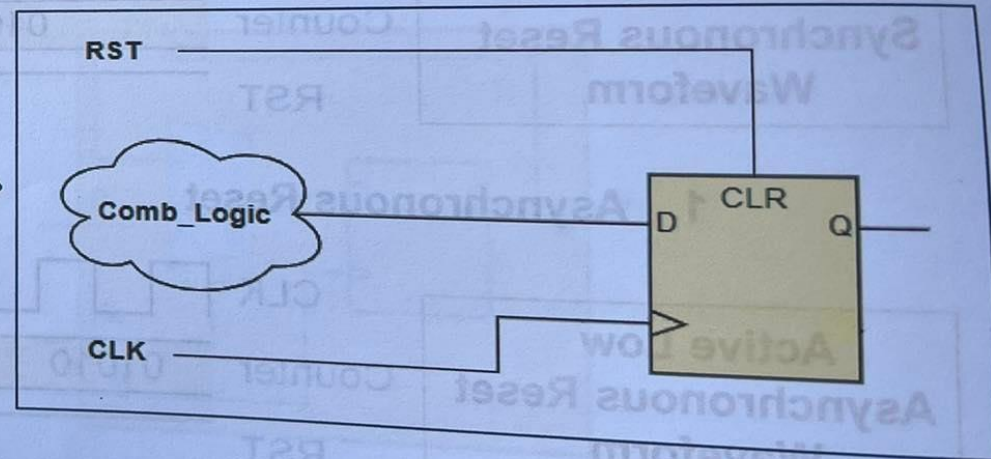
## - Active Low Asynchronous Reset

```
always @ (posedge CLK or negedge RST)
if ( ! RST)
    Q <= 1'b0 ;
else
    Q <= D
assign D = Comb_Logig
```



## - Active High Asynchronous Reset

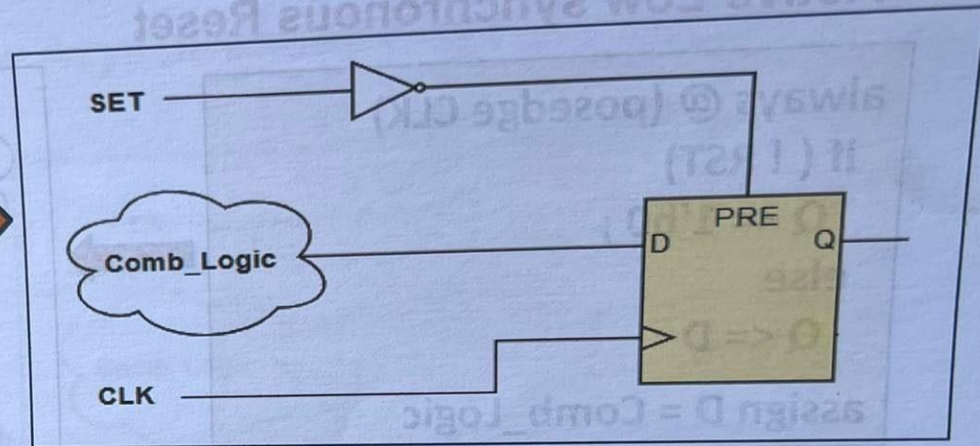
```
always @ (posedge CLK or posedge RST)
if ( RST)
    Q <= 1'b0 ;
else
    Q <= D
assign D = Comb_Logig
```



## D Flip Flop With Asynchronous Preset

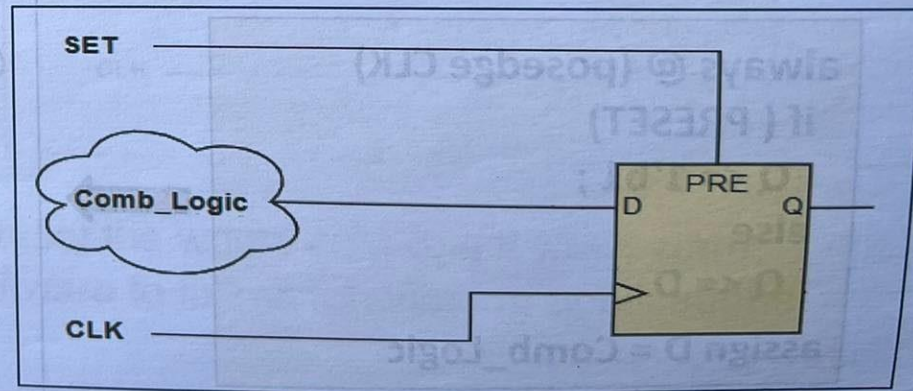
### - Active Low Asynchronous Preset

```
always @ (posedge CLK or negedge PRESET)
if ( ! PRESET)
    Q <= 1'b1 ;
else
    Q <= D
assign D = Comb_Logical
```



### - Active High Asynchronous Preset

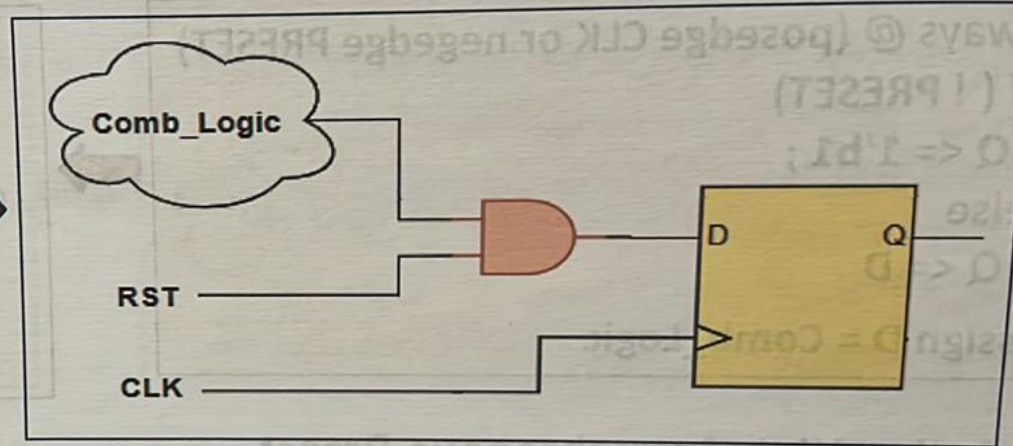
```
always @ (posedge CLK or posedge PRESET)
if (PRESET)
    Q <= 1'b1 ;
else
    Q <= D
assign D = Comb_Logical
```



## D Flip Flop With synchronous Reset/Preset

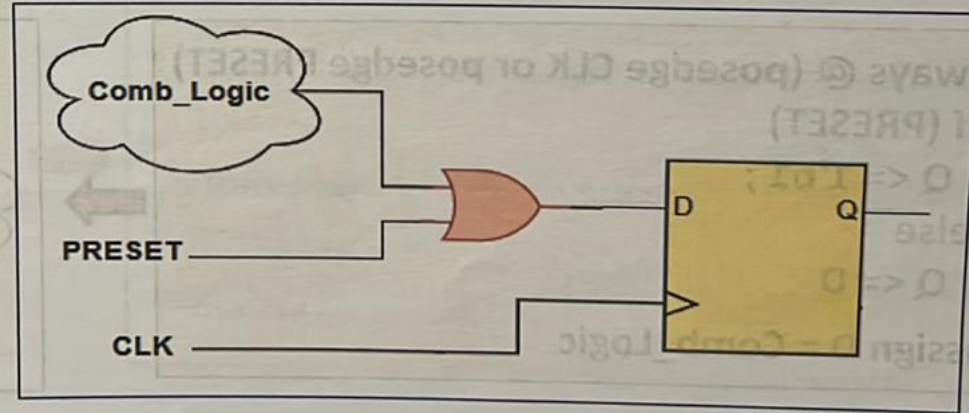
### - Active Low synchronous Reset

```
always @ (posedge CLK)
  if ( ! RST)
    Q <= 1'b0 ;
  else
    Q <= D
assign D = Comb_Logic
```



### - Active High synchronous Preset

```
always @ (posedge CLK)
  if ( PRESET)
    Q <= 1'b1 ;
  else
    Q <= D
assign D = Comb_Logic
```

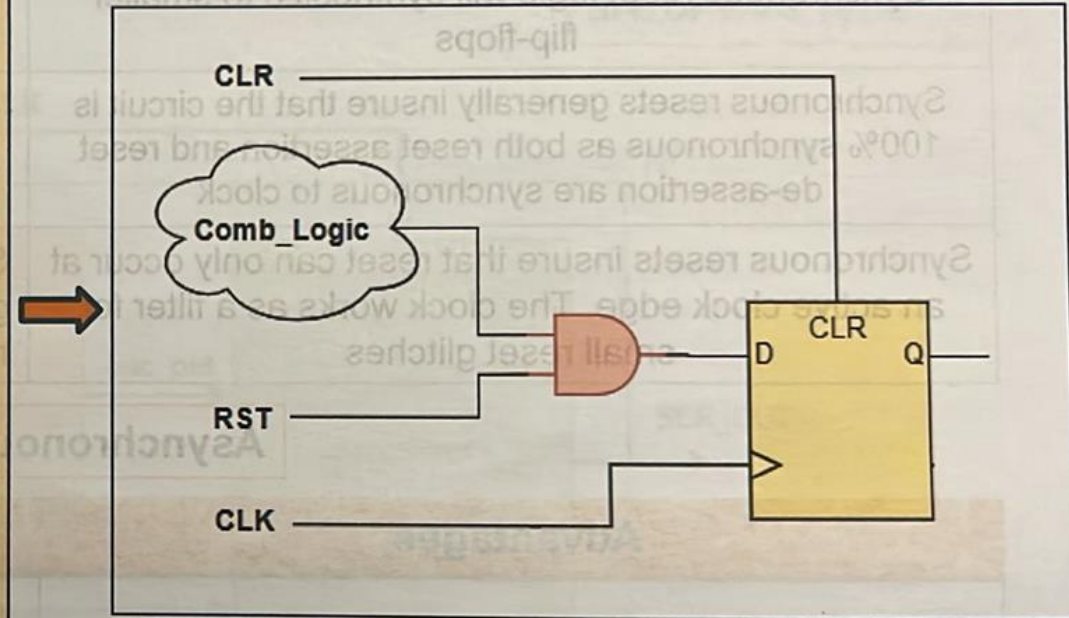


## D Flip Flop With Asyn/Sync Reset

- Active Low synchronous reset & Active Low Asynchronous reset

```
always @ (posedge CLK or negedge Hard_RST)
if ( ! Hard_RST)
    Q <= 1'b0 ;
else if ( ! Soft_RST)
    Q <= 1'b0 ;
else
    Q <= D

assign D = Comb_Logic
```



**Hard reset** is a global reset used to initialize or reset the whole chip to a known state, while **Soft reset** is initiated from the **devices** software to fix malfunctioning applications.

# Flip Flop Reset

## Synchronous Reset

| Advantages                                                                                                                                       | Disadvantages                                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Synchronous reset logic will synthesize to smaller flip-flops                                                                                    | Not all ASIC libraries have flip-flops with built-in synchronous resets                                                                                   |
| Synchronous resets generally insure that the circuit is 100% synchronous as both reset assertion and reset de-assertion are synchronous to clock | Add extra overhead in logic cone between FF's which limit the speed of design and increase gate count.                                                    |
| Synchronous resets insure that reset can only occur at an active clock edge. The clock works as a filter for small reset glitches                | Synchronous resets may need a pulse stretcher to guarantee a reset pulse width wide enough to ensure reset is present during an active edge of each clock |

## Asynchronous Reset

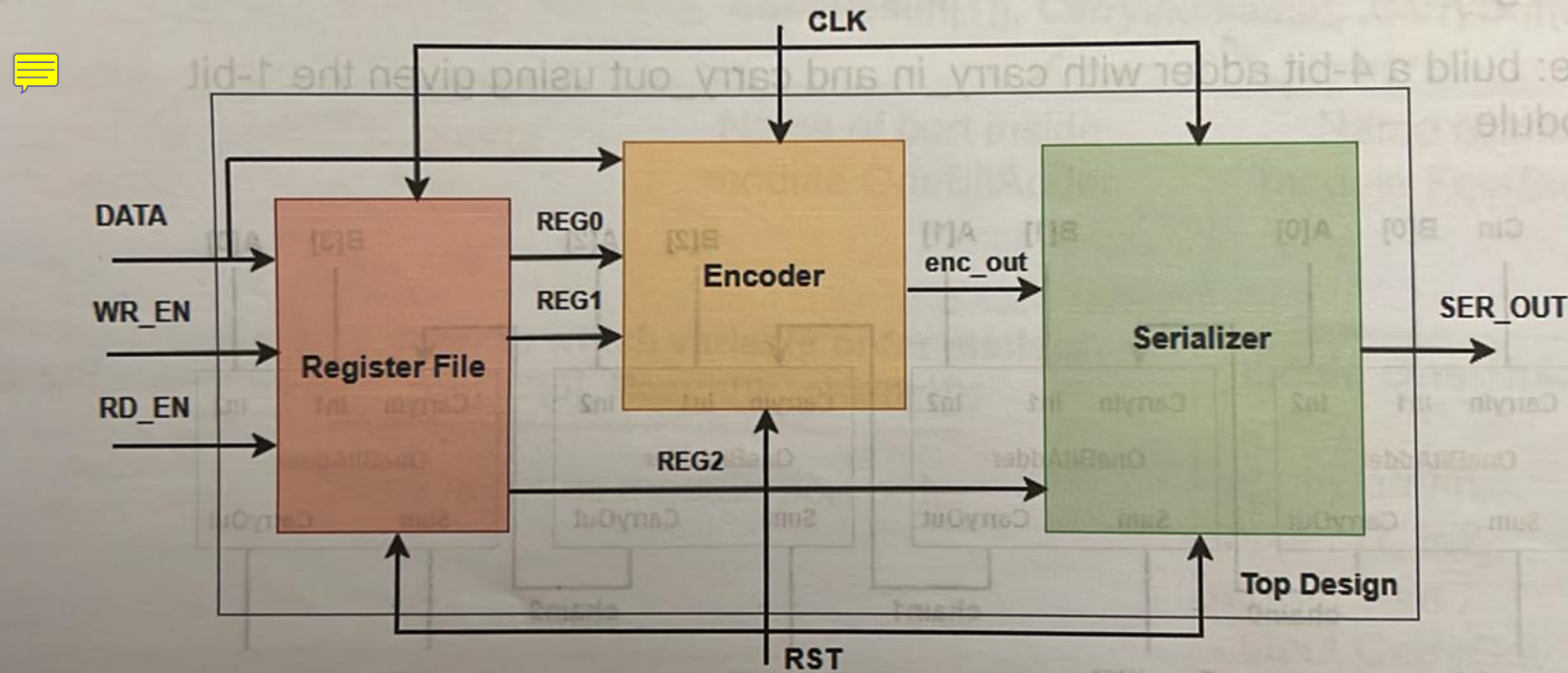
| Advantages                                                                                      | Disadvantages                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| All ASIC libraries has asynchronously reset-able flip-flops which not limit the speed of design | If the asynchronous reset is released at or near the active clock edge of a flip-flop, the output of the flip-flop could go metastable "solved by <b>reset synchronizer</b> " |
| Asynchronous resets is a circuit can be reset with or without a clock present.                  | The asynchronous reset line is subject to glitching which need Reset-glitch filtering circuit " <b>delay circuit</b> " to filter out small glitches to avoid unwanted reset   |

# Structural Verilog

Structural Verilog is used to :-

- 1) Assemble several blocks together in a hierarchical design

Internal connections  
are of wire Type

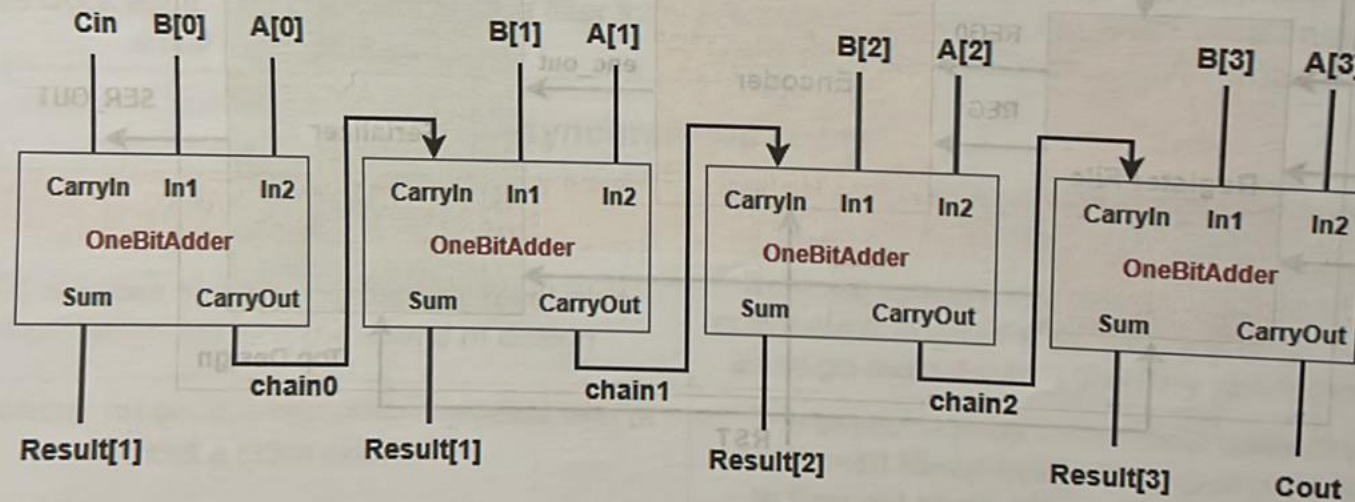


# Structural Verilog

Structural Verilog is used to :-

2) Build complex modules by 'building' (instantiating) a number of smaller modules and put them together

Example: build a 4-bit adder with carry\_in and carry\_out using given the 1-bit adder module



# Structural Verilog

## Module instance format

*name*  
// Port Mapping by position in which variable order does not matter  
**OneBitAdder u1** (.In1(A[1]), .In2(B[1]), .Sum(Result[1]), CarryIn(chain0), .CarryOut(chain1));

module  
name

Instance  
Name

Name of port inside  
module OneBitAdder

Name of wire in  
module FourBitAdder

// Port Mapping by order in which variable order matters  
**OneBitAdder u1** (Cin, A[0], B[0], Result[0], chain[1]);

Same order as module port order

```
module OneBitAdder
(
    input CarryIn,
    input In1, In2,
    output sum ,
    output CarryOut
);
```

# Parameterized Design

## Parameter

- Verilog parameter is used to define a constant value within the module structure.
- Module parameters can be overwritten during module instantiation.
- It allow the code to be parameterized and reusable

### Parameter definition

```
module RegFile #( parameter WIDTH = 16 , parameter DEPTH = 64 )  
( input [WIDTH-1:0] DATA // port declaration );
```

### Parameter overwrite

```
RegFile #( .WIDTH(8) , .DEPTH (8) ) U1 (port mapping) ;
```

### Parameterized Variable Assignment

```
assign DATA = 'b1110011 ;
```

## Localparam

- Verilog localparam is used to define a constant value **local** within the module structure.
- localparam can't be overwritten during module instantiation.
- The idea behind the localparam is to protect it's value from accidental redefinition by an end-user.

### Parameter definition

```
localparam WIDTH = 16 ;
```

### Variable Assignment

```
assign DATA = 'b1110011 ;
```

## Macros

- Verilog Macro is a global parameter, modified once and can be used in all the modules.

### Macro definition

```
`define WIDTH 16
```

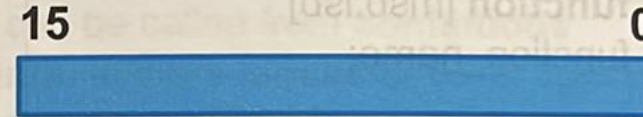
### Macros Assignment

```
assign DATA = `WIDTH ;
```

# Multidimensional Array

## 1D Vector

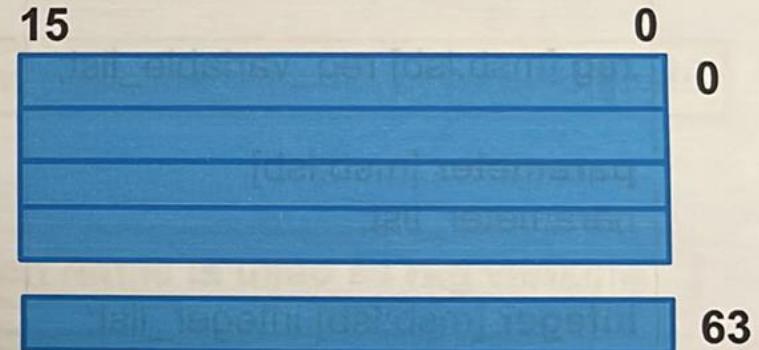
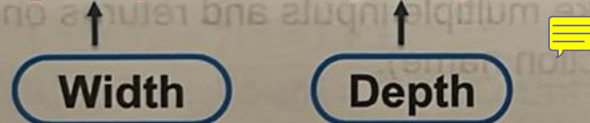
- Declare 16-bit vector signal named DATA
- ```
reg [15:0] DATA;
```



## 2D Vector

- Declare an array of 64 X 16-bit vectors named MEM

```
reg [15:0] MEM [0:63];
```



Note arrays can NOT be passed through input/output ports